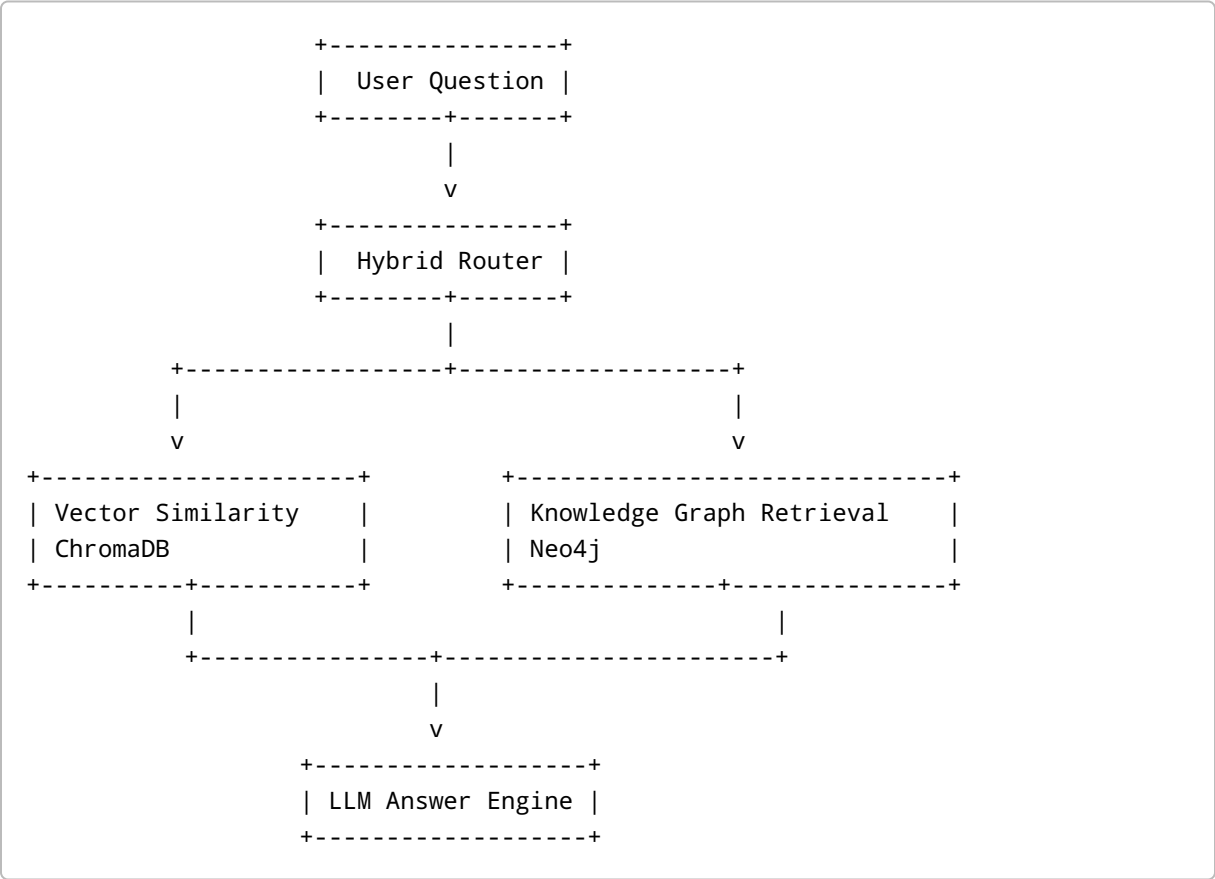


GraphRAG Upgrade for RAG-Powered-Q-A-Chatbot-main

This upgrade converts your basic RAG chatbot into an enterprise-style GraphRAG chatbot using:

- Knowledge Graph extraction
- Neo4j graph database
- Hybrid Retrieval (Vector + Graph)
- Entity-aware Q&A
- LangChain + GraphCypherQChain
- Streamlit UI
- OpenAI / Ollama compatible architecture

Recommended Architecture



New Project Structure

```
RAG-Powered-Q-A-Chatbot-main/  
|  
├─ app.py  
├─ graph_rag_chain.py  
├─ graph_builder.py  
├─ graph_retriever.py  
├─ vector_store.py  
├─ document_processor.py  
├─ requirements.txt  
├─ .env  
|  
├─ data/  
|   └─ documents/  
|  
├─ chroma_db/  
|  
└─ graph_db/
```

1. Updated requirements.txt

```
langchain  
langchain-community  
langchain-openai  
langchain-experimental  
openai  
chromadb  
sentence-transformers  
streamlit  
pypdf  
pymupdf  
python-dotenv  
tiktoken  
neo4j  
networkx  
transformers  
torch  
spacy  
numpy
```

```
pandas
faiss-cpu
```

2. graph_builder.py

This extracts entities and relationships from documents and stores them in Neo4j.

```
import os
import spacy
from neo4j import GraphDatabase
from dotenv import load_dotenv
from document_processor import DocumentProcessor

load_dotenv()

class GraphBuilder:
    def __init__(self):
        self.nlp = spacy.load("en_core_web_sm")

        self.driver = GraphDatabase.driver(
            os.getenv("NEO4J_URI"),
            auth=(
                os.getenv("NEO4J_USERNAME"),
                os.getenv("NEO4J_PASSWORD")
            )
        )

        self.processor = DocumentProcessor()

    def close(self):
        self.driver.close()

    def extract_entities_relations(self, text):
        doc = self.nlp(text)

        entities = []
        relations = []

        for ent in doc.ents:
            entities.append({
                "name": ent.text,
                "label": ent.label_
            })
```

```

sentences = list(doc.sents)

for sent in sentences:
    sent_doc = self.nlp(sent.text)
    ents = [e.text for e in sent_doc.ents]

    if len(ents) >= 2:
        for i in range(len(ents)-1):
            relations.append({
                "source": ents[i],
                "target": ents[i+1],
                "relation": "RELATED_TO"
            })

return entities, relations

def create_graph(self, documents_path):
    documents = self.processor.process_documents(documents_path)

    with self.driver.session() as session:
        for doc in documents:
            entities, relations = self.extract_entities_relations(
                doc.page_content
            )

            for entity in entities:
                session.run(
                    """
                    MERGE (e:Entity {name: $name})
                    SET e.label = $label
                    """,
                    name=entity["name"],
                    label=entity["label"]
                )

            for rel in relations:
                session.run(
                    """
                    MATCH (a:Entity {name: $source})
                    MATCH (b:Entity {name: $target})
                    MERGE (a)-[:RELATED_TO]->(b)
                    """,
                    source=rel["source"],
                    target=rel["target"]
                )

    print("Knowledge graph created successfully")

```

```
if __name__ == "__main__":
    builder = GraphBuilder()
    builder.create_graph("data/documents")
    builder.close()
```

3. graph_retriever.py

Hybrid Graph Retrieval.

```
import os
from neo4j import GraphDatabase
from dotenv import load_dotenv

load_dotenv()

class GraphRetriever:
    def __init__(self):
        self.driver = GraphDatabase.driver(
            os.getenv("NEO4J_URI"),
            auth=(
                os.getenv("NEO4J_USERNAME"),
                os.getenv("NEO4J_PASSWORD")
            )
        )

    def close(self):
        self.driver.close()

    def query_graph(self, question):
        keywords = question.split()

        cypher_query = """
        MATCH (a)-[r]->(b)
        WHERE any(keyword IN $keywords
            WHERE toLower(a.name) CONTAINS toLower(keyword)
            OR toLower(b.name) CONTAINS toLower(keyword))
        RETURN a.name as source,
            type(r) as relation,
            b.name as target
        LIMIT 10
        """
```

```

with self.driver.session() as session:
    result = session.run(
        cypher_query,
        keywords=keywords
    )

    data = []
    for row in result:
        data.append(
            f"{row['source']} -[{row['relation']}]> {row['target']}"
        )

    return "\n".join(data)

```

4. Updated vector_store.py

```

from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from document_processor import DocumentProcessor

class VectorStore:
    def __init__(self, persist_directory="./chroma_db"):
        self.persist_directory = persist_directory

        self.embeddings = HuggingFaceEmbeddings(
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )

        self.processor = DocumentProcessor()

    def build_vector_store(self, docs_path):
        docs = self.processor.process_documents(docs_path)

        db = Chroma.from_documents(
            documents=docs,
            embedding=self.embeddings,
            persist_directory=self.persist_directory
        )

        db.persist()

        print("Vector DB created")

```

```

def get_retriever(self):
    db = Chroma(
        persist_directory=self.persist_directory,
        embedding_function=self.embeddings
    )

    return db.as_retriever(search_kwargs={"k": 4})

```

5. graph_rag_chain.py

Main GraphRAG pipeline.

```

import os
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import RetrievalQA

from vector_store import VectorStore
from graph_retriever import GraphRetriever

load_dotenv()

class GraphRAGChain:
    def __init__(self):
        self.vector_store = VectorStore()
        self.retriever = self.vector_store.get_retriever()

        self.graph_retriever = GraphRetriever()

        self.llm = ChatOpenAI(
            model="gpt-4o-mini",
            temperature=0
        )

        self.prompt_template = """
You are an enterprise GraphRAG assistant.

Use BOTH:
1. Vector document context
2. Knowledge graph relationships

```

to answer the question.

Vector Context:
{context}

Knowledge Graph Context:
{graph_context}

Question:
{question}

Answer:
"""

```
        self.prompt = PromptTemplate(
            template=self.prompt_template,
            input_variables=[
                "context",
                "graph_context",
                "question"
            ]
        )

    def query(self, question):
        docs = self.retriever.get_relevant_documents(question)

        vector_context = "\n\n".join([
            d.page_content for d in docs
        ])

        graph_context = self.graph_retriever.query_graph(question)

        final_prompt = self.prompt.format(
            context=vector_context,
            graph_context=graph_context,
            question=question
        )

        response = self.llm.invoke(final_prompt)

    return {
        "answer": response.content,
        "vector_context": vector_context,
```



```
        "graph_context": graph_context
    }
```

6. Updated app.py

```
import streamlit as st
from graph_rag_chain import GraphRAGChain

st.set_page_config(
    page_title="GraphRAG Chatbot",
    layout="wide"
)

@st.cache_resource
def load_chain():
    return GraphRAGChain()

chain = load_chain()

st.title("Enterprise GraphRAG Q&A Chatbot")

if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

question = st.chat_input("Ask a question")

if question:
    st.session_state.messages.append(
        {
            "role": "user",
            "content": question
        }
    )
```

```

with st.chat_message("user"):
    st.markdown(question)

with st.chat_message("assistant"):
    with st.spinner("Thinking..."):
        result = chain.query(question)

        st.markdown(result["answer"])

        with st.expander("Graph Context"):
            st.code(result["graph_context"])

        with st.expander("Vector Context"):
            st.code(result["vector_context"][:3000])

st.session_state.messages.append(
    {
        "role": "assistant",
        "content": result["answer"]
    }
)

```

7. Updated document_processor.py

```

from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import (
    PyPDFLoader,
    TextLoader
)

import os

class DocumentProcessor:
    def __init__(self):
        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=1000,
            chunk_overlap=200
        )

    def load_documents(self, docs_path):
        docs = []

        for file in os.listdir(docs_path):

```

```
path = os.path.join(docs_path, file)

if file.endswith(".pdf"):
    loader = PyPDFLoader(path)
    docs.extend(loader.load())

elif file.endswith(".txt"):
    loader = TextLoader(path)
    docs.extend(loader.load())

return docs

def process_documents(self, docs_path):
    docs = self.load_documents(docs_path)
    return self.splitter.split_documents(docs)
```

8. .env

```
OPENAI_API_KEY=your_openai_key

NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=password
```

9. Neo4j Docker Setup

```
docker run
  --name neo4j
  -p7474:7474 -p7687:7687
  -d
  -e NEO4J_AUTH=neo4j/password
  neo4j:latest
```

Neo4j Browser:

```
http://localhost:7474
```

10. Build the Graph + Vector DB

```
python graph_builder.py
```

```
from vector_store import VectorStore

vs = VectorStore()
vs.build_vector_store("data/documents")
```

11. Run the Application

```
streamlit run app.py
```

12. Advanced Enterprise Enhancements

A. Graph Embeddings

Add:

- Node2Vec
- GraphSAGE
- Neo4j Graph Data Science

for semantic relationship retrieval.

B. Multi-Hop Reasoning

Example:

```
MATCH p=(a)-[*1..3]->(b)
RETURN p
```

This allows:

- fraud network analysis

- root cause analysis
 - dependency tracing
 - enterprise impact analysis
-

C. Hybrid Search Ranking

Combine:

```
Final Score =  
0.7 * Vector Similarity  
+  
0.3 * Graph Relationship Score
```

D. Production Improvements

Recommended:

- Redis cache
 - Async retrieval
 - Streaming responses
 - OpenTelemetry tracing
 - Kubernetes deployment
 - RBAC security
 - Document lineage
 - Prompt versioning
 - Semantic caching
-

13. Suggested Enterprise GraphRAG Use Cases

This architecture is suitable for:

- Fraud Detection Investigation
- AML Entity Relationship Analysis
- Banking Knowledge Assistant
- Insurance Claim Investigation
- Enterprise Document Search
- Legal Contract Intelligence
- IT Incident Dependency Mapping
- Healthcare Knowledge Graphs
- Customer Support Copilot

14. Migration from Current RAG → GraphRAG

Current RAG	GraphRAG
Vector Search Only	Vector + Graph Search
Flat Chunks	Entity Relationships
Semantic Similarity	Multi-hop Reasoning
Limited Context	Connected Knowledge
Basic Q&A	Relationship-aware Q&A
No Entity Intelligence	Enterprise Knowledge Graph

15. Recommended Future Enhancements

Add Ollama Support

Replace:

```
from langchain_openai import ChatOpenAI
```

with:

```
from langchain_community.chat_models import ChatOllama
```

Example:

```
self.llm = ChatOllama(  
    model="llama3"  
)
```

Add PDF + Image OCR

Use:

- Tesseract OCR

- LayoutParser
 - Unstructured.io
-

Add Agentic GraphRAG

Introduce:

- Planner Agent
 - Retrieval Agent
 - Graph Agent
 - Citation Agent
 - Validation Agent
-

Final Result

After implementing these changes, your chatbot becomes:

- Graph-aware
- Multi-hop reasoning capable
- Enterprise scalable
- Neo4j powered
- Hybrid semantic search enabled
- Relationship-aware
- Better than traditional RAG